

- [Excel Parser System Documentation](#)
 - [Table of Contents](#)
 - [Getting Started](#)
 - [What Is This System?](#)
 - [The Big Picture](#)
 - [System Components Explained](#)
 - [Quick Example](#)
 - [How It All Works Together](#)
 - [Step-by-Step Flow](#)
 - [Step 1: You Make a Request](#)
 - [Step 2: System Reads the Map \(config.json\)](#)
 - [Step 3: System Opens Excel File](#)
 - [Step 4: System Finds the Data](#)
 - [Visual Flow Diagram](#)
 - [Understanding config.json](#)
 - [What Is config.json?](#)
 - [Why Use a Config File?](#)
 - [Basic Structure Explained](#)
 - [Real-World Analogy](#)
 - [Structure](#)
 - [Understanding Sheet Types](#)
 - [Type 1: Letters Price Sheets \(type: "letters_price"\)](#)
 - [Configuration Fields Explained](#)
 - [Visual Example with Real Numbers](#)
 - [Type 2: Bars Sheets \(type: "bars"\)](#)
 - [Configuration Fields Explained](#)
 - [Visual Example](#)
 - [Type 3: Logos Sheets \(type: "logos"\)](#)
 - [Configuration Fields Explained](#)
 - [Visual Example](#)
 - [For Upcharge Files](#)
 - [Understanding FlexibleExcelParser.py](#)
 - [What Does This File Do?](#)
 - [The Main Function: get_price\(\)](#)
 - [Function Signature](#)
 - [Parameters Explained \(Like Talking to a Friend\)](#)
 - [What Happens Inside \(Step-by-Step\)](#)

- Return Value
- The Parser Functions (The Actual Workers)
 - Parser 1: `parse_letters()` - Finding Letter Prices
 - Parser 2: `parse_bars()` - Finding Bar Prices
 - Parser 3: `parse_logos()` - Finding Logo Prices
 - 2. `parse_bars(df, cfg, params)`
 - 3. `parse_logos(df, cfg, params)`
- Understanding `upcharges.py`
 - What Does This File Do?
 - Why Separate from Main Parser?
 - The Main Function: `get_upcharge()`
 - Function Signature
 - Parameters Explained
 - Return Values
 - Detailed Step-by-Step Process
 - Excel Structure
 - Matching Logic
- Complete Usage Examples
 - Example 1: Get a Simple Letter Price
 - Example 2: Get a Bar Price
 - Example 3: Get a Logo Price
 - Example 4: Get an Upcharge
 - Example 5: Complete Price Calculation (Real-World Scenario)
 - Example 6: Error Handling
 - Example 7: Working with Multiple Products
 - Example 8: Batch Processing
- Common Issues and Solutions
 - Issue 1: "Country not found"
 - Issue 2: "Thickness not found"
 - Issue 3: "Logo thickness label not found"
 - Issue 4: Upcharge returns None
 - Issue 5: Column index out of range
- Configuration Best Practices
- File Dependencies
- Notes
- Quick Reference Guide
 - For Beginners: What Do I Need to Know?
 - 1. The Three Main Concepts

- [2. Quick Decision Tree](#)
- [3. Common Parameter Patterns](#)
- [4. Key Things to Remember](#)
- [5. When Things Go Wrong](#)
- [6. Understanding Return Values](#)
- [7. Complete Example Template](#)
- [Summary](#)
- [Next Steps](#)

Excel Parser System Documentation

Table of Contents

1. [Getting Started](#)
 2. [How It All Works Together](#)
 3. [Understanding config.json](#)
 4. [Understanding FlexibleExcelParser.py](#)
 5. [Understanding upcharges.py](#)
 6. [Complete Usage Examples](#)
 7. [Troubleshooting Guide](#)
-

Getting Started

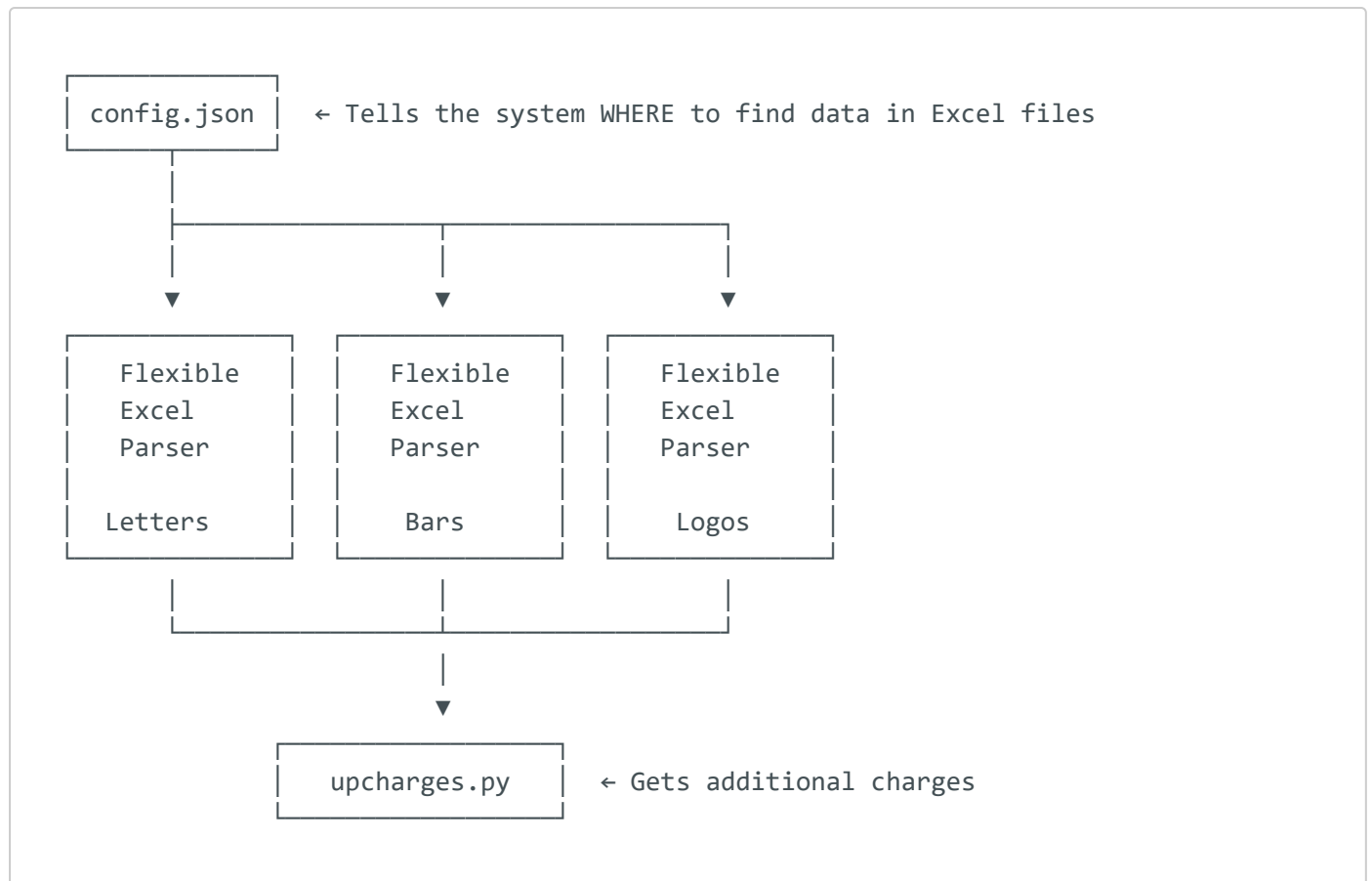
What Is This System?

Imagine you have Excel files with pricing information, but the data is organized in different ways:

- Some sheets have prices organized by country (US/Canada) and thickness
- Some have prices in tables based on dimensions (depth × height)
- Some have prices in 2D grids based on square inches
- Some have additional "upcharges" for special finishes or options

This system automatically reads these Excel files and extracts the exact price you need, without you having to manually search through spreadsheets.

The Big Picture



System Components Explained

1. **config.json** - The "Map"

- Think of this as a map that tells the system:
 - Which Excel file to open
 - Which sheet to look at
 - Where in that sheet the data is located
 - How the data is organized

2. **FlexibleExcelParser.py** - The "Price Finder"

- This reads the config map
- Opens the correct Excel file
- Finds the exact price you're looking for
- Returns the price value

3. **upcharges.py** - The "Extra Charges Finder"

- Similar to the parser, but specifically for upcharges
- Handles special cases like "BASE PRICE"
- Returns additional charges for finishes, mounting options, etc.

Quick Example

Let's say you want to know: "What's the price for a 1/8 inch thick letter, size 3, in the US?"

```
# You call the function:
price = get_price(
    category="FLAT_CUT_PVC",
    sheet_name="Letters PVC",
    params={"country": "US", "thickness": "1/8", "column": 3}
)

# The system:
# 1. Looks in config.json to find "FLAT_CUT_PVC" → "Letters PVC" sheet
# 2. Opens the Excel file specified in config
# 3. Finds the "US" section
# 4. Looks for row with thickness "1/8"
# 5. Gets the value from column 3
# 6. Returns: 22.50 (for example)
```

How It All Works Together

Step-by-Step Flow

Let's trace through what happens when you request a price:

Step 1: You Make a Request

```
get_price("FLAT_CUT_PVC", "Letters PVC", {"country": "US", "thickness": "1/8",
"column": 2})
```

Step 2: System Reads the Map (config.json)

config.json says:

- Category "FLAT_CUT_PVC" uses file "flat_cut_pvc .xlsx"
- Sheet "Letters PVC" has type "letters_price"
- Country labels are in column 0 (Column A)
- Data starts 2 rows after country label
- Prices start in column 1 (Column B)

Step 3: System Opens Excel File

Opens: flat_cut_pvc .xlsx

Reads: Sheet named "Letters PVC"

Step 4: System Finds the Data

Looks in Column A for "US"

Finds "US" at row 1

Skips 2 rows → starts reading at row 3

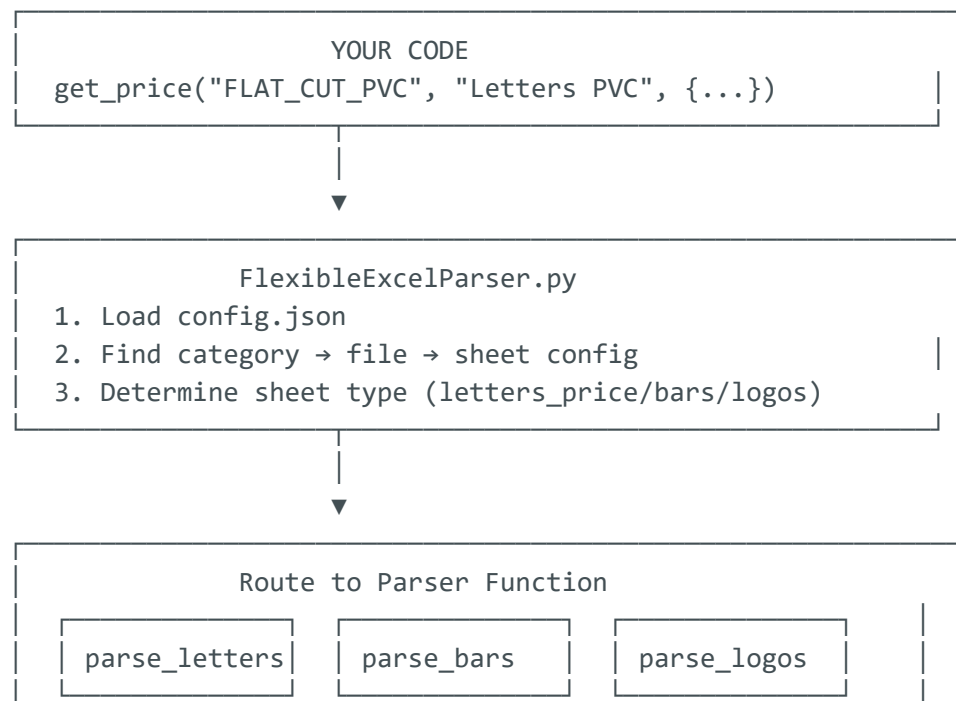
Looks for thickness "1/8" in Column A

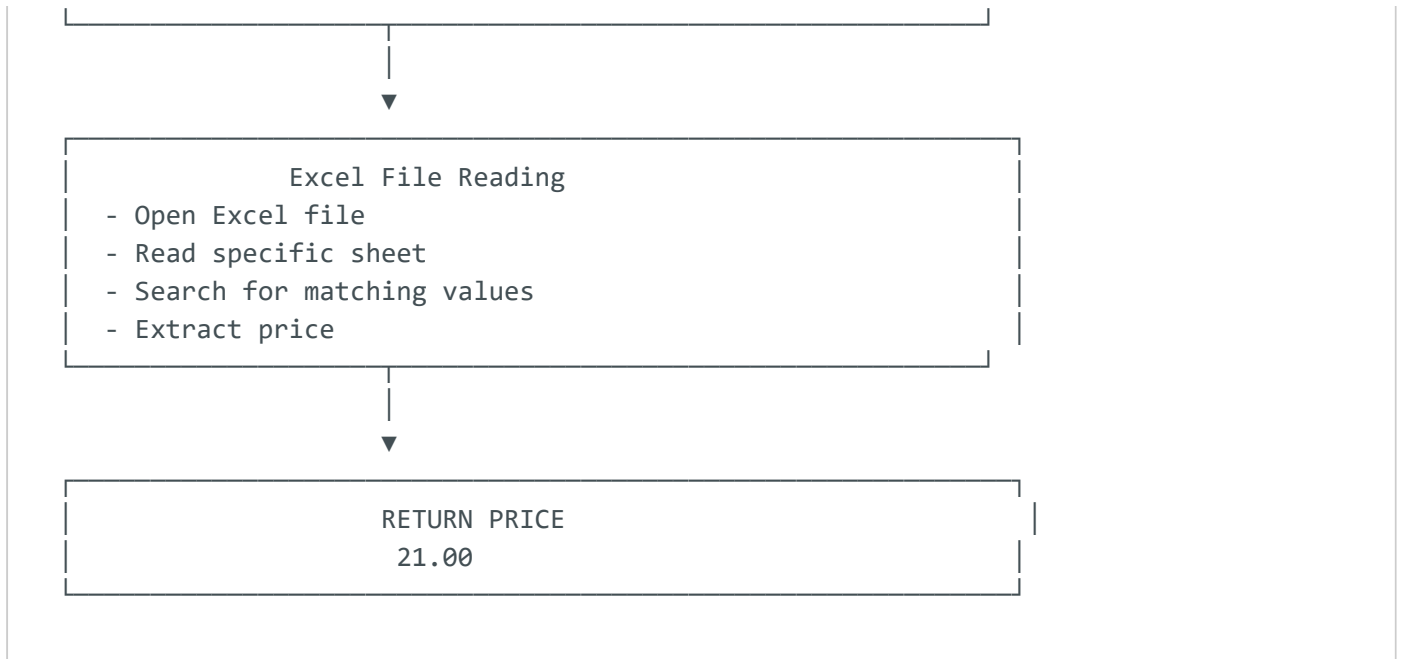
Finds "1/8" at row 3

Gets value from Column C (column index 2)

Returns: 21.00

Visual Flow Diagram





Understanding config.json

What Is config.json?

Think of **config.json** as a **detailed instruction manual** for reading Excel files. Instead of hardcoding where data is located in your Python code, you define it once in this JSON file. This makes it easy to:

- Add new product lines without changing code
- Update file locations in one place
- Understand the Excel structure at a glance

Why Use a Config File?

Without config.json (Bad):

```
# Hardcoded - hard to maintain
df = pd.read_excel("flat_cut_pvc.xlsx", sheet_name="Letters PVC")
price = df.iloc[3, 2] # What does row 3, column 2 mean??
```

With config.json (Good):

```
# Flexible - easy to understand and change
config = load_config()
# Config tells us exactly where to look
price = get_price_using_config(config, params)
```

Basic Structure Explained

Let's break down what each part means:

```
{
  "file_structures": {                                // All product categories
    "FLAT_CUT_PVC": {                                  // One product category
      "file_pattern": "flat_cut_pvc.xlsx",            // Which Excel file to use
      "productlineFile": {                             // Sheets in that file
        "Letters PVC": {                               // One sheet
          "type": "letters_price",                     // What kind of data is here
          "country_column": 0,                         // Where to find country labels
          // ... more instructions
        }
      }
    }
  }
}
```

Real-World Analogy

Imagine you're giving directions to find a house:

- **file_pattern**: "Go to 123 Main Street" (which file)
- **sheet_name**: "Apartment 2B" (which sheet)
- **country_column**: "Look at the front door" (where to start)
- **data_start_offset**: "Go up 2 floors" (how many rows to skip)
- **price_start_column**: "Check room number 3" (which column has prices)

Structure

```
{
  "file_structures": {
    "CATEGORY_NAME": {
      "productLineName": "Display Name",
```



```

"productLineKey": "url-key",
"file_pattern": "filename.xlsx",
"productlineFile": {
  "product_name": {
    "file_pattern": "product_file.xlsx",
    "sheets": {
      "Sheet Name": {
        "type": "letters_price|bars|logos",
        // ... sheet-specific config
      }
    }
  }
},
"upchargeFile": "upcharge_file.xlsx",
"upchargeFileStructure": {
  "blocks": {
    "Material Name": {
      "label": 0,
      "type": 1,
      "us": 2,
      "canada": 3
    }
  }
}
}
}
}

```

Understanding Sheet Types

The system supports three main types of price sheets. Each type has a different structure, so the config tells the parser how to read each one.

Type 1: Letters Price Sheets (**type: "letters_price"**)

What it's for: Prices for individual letters, organized by country and thickness.

How it works: The Excel sheet has sections for each country (US, Canada), and within each section, rows represent different thicknesses, with columns representing different sizes.

Configuration Fields Explained

Field	What It Means	Why It Matters	Example
country_column	Which column has "US" or "Canada" labels	Tells parser where to look for country sections	0 (Column A)
header_row_offset	How many rows down from country label to find size headers (1, 2, 3, etc.)	Headers tell us what each column represents	1 (1 row down)
data_start_offset	How many rows down from country label to start reading prices	Data rows come after the header row	2 (2 rows down)
thickness_column	Which column has thickness values (1/8, 1/4, etc.)	We match the requested thickness here	0 (Column A)
price_start_column	Which column has the first price	Prices start here and continue right	1 (Column B)
stop_conditions	What values mean "stop reading"	Prevents reading into next country's data	["US", "Canada", "nan", ""]
country_labels	Valid country names to search for	Only these will be recognized	["US", "Canada"]

Visual Example with Real Numbers

Let's see how the parser reads this Excel structure:

Excel Sheet: "Letters PVC"					
	Column A	Column B	Column C	Column D	Column E
Row 0:	[US,	-,	-,	-,	...]
	↑				
	Country label found here (country_column = 0)				
Row 1:	[-,	1,	2,	3,	4, ...]
	↑	↑			
	Skip this	Headers start here (header_row_offset = 1)			

(price_start_column = 1)

```
Row 2:  [1/8,      20.00,      21.00,      22.00,      23.00, ...]
        ↑          ↑
        Thickness  Price for size 1
        (thickness_column = 0) (column 0 = size 1, column 1 = size 2, etc.)

Row 3:  [1/4,      20.00,      21.00,      22.00,      23.00, ...]
Row 4:  [3/8,      25.00,      26.00,      27.00,      28.00, ...]
Row 5:  [1/2,      30.00,      31.00,      32.00,      33.00, ...]
...
Row 8:  [Canada,   -,         -,         -,         ...]
        ↑
        Stop! (stop_condition found)
```

Step-by-Step Reading Process:

1. Find Country Section:

- Parser searches Column A (country_column = 0) for "US"
- Finds "US" at Row 0
- `start_row = 0`

2. Find Headers:

- Headers are at: `start_row + header_row_offset = 0 + 1 = Row 1`
- Reads: `[1, 2, 3, 4, ...]` starting from Column B (price_start_column = 1)

3. Start Reading Data:

- Data starts at: `start_row + data_start_offset = 0 + 2 = Row 2`
- Reads Row 2: `[1/8, 20.00, 21.00, 22.00, ...]`

4. Find Requested Thickness:

- Looking for thickness "1/8"
- Checks Column A (thickness_column = 0) of each row
- Finds "1/8" at Row 2

5. Get Price:

- Requested column = 2 (which means size 3)
- Price is at: `Row 2, Column (price_start_column + column_index) = Column 3`
- Returns: `22.00`

6. Stop When:

- Reaches "Canada" in Column A (stop_condition)
- Or reaches end of file

Type 2: Bars Sheets (type: "bars")

What it's for: Prices for bars (rectangular pieces), organized by depth and height dimensions.

How it works: The Excel sheet has separate blocks for US and Canada data side-by-side. Each block has a label (like "US 1/8"), then a table where rows are depths and columns are heights.

Configuration Fields Explained

Field	What It Means	Why It Matters	Example
us_columns	Column range [start, end] for US data block	US and Canada data are in different columns	[0, 7] (Columns A-G)
canada_columns	Column range [start, end] for Canada data block	Canada data starts where US ends	[7, 13] (Columns H-M)
header_row_offset	Rows down from label to find height headers	Headers tell us height values	1
data_start_offset	Rows down from label to start reading depth/price data	Data rows come after headers	2

Visual Example

Excel Sheet: "Bars-Aluminum"

US Block (Columns 0-6)

Column A	B	C	D	E	F
----------	---	---	---	---	---

Canada Block (Columns 7-12)

Column H	I	J	K	L	M
----------	---	---	---	---	---

Row 0: US 1/8"
Row 1: - 1 2 3 4 5
Row 2: 12 10 11 12 13 14
Row 3: 24 20 21 22 23 24
Row 4: 36 30 31 32 33 34

↑ ↑ ↑ ↑ ↑ ↑
Label Heights (headers)
↓
Depths (row values)

Row 0: Canada 1/8"
Row 1: - 1 2 3 4 5
Row 2: 12 15 16 17 18 19
Row 3: 24 25 26 27 28 29
Row 4: 36 35 36 37 38 39

↑ ↑ ↑ ↑ ↑ ↑
Label Heights (headers)

Reading Process:

1. Determine Block:

- Request: `label = "US 1/8\""`
- Starts with "US" → use `us_columns = [0, 7]`
- Extract columns 0-6 (US block)

2. Find Label:

- Search Column A (first column of US block) for "US 1/8"
- Finds at Row 0
- `start_row = 0`

3. Read Heights:

- Headers at: `start_row + header_row_offset = 0 + 1 = Row 1`
- Reads: `[1, 2, 3, 4, 5]` from columns B-F

4. Find Depth:

- Request: `depth = 36`
- Start reading from: `start_row + data_start_offset = 0 + 2 = Row 2`
- Check Column A of each row for depth value
- Finds `36` at Row 4

5. Get Price:

- Request: `height = 3`
- Height 3 is at index 2 in headers `[1, 2, 3, 4, 5]`
- Price column = `1 + 2 = Column C` (index 2)
- Returns: `32.00` (from Row 4, Column C)

Type 3: Logos Sheets (type: "logos")

What it's for: Prices for logos based on square inch dimensions. Uses a 2D lookup table.

How it works: The Excel sheet has sections for each thickness (1/8 Inch, 1/4 Inch, etc.). Each section is a table where:

- **Row headers** = one dimension in square inches (10, 11, 12, ...)
- **Column headers** = another dimension in square inches (10, 11, 12, ...)
- **Cell value** = price for that combination

Configuration Fields Explained

Field	What It Means	Why It Matters	Example
thickness_labels	List of valid thickness section labels	Parser searches for these to find the right section	["1/8 Inch", "1/4 Inch", ...]
thickness_column	Column where thickness labels appear	Where to look for section markers	0 (Column A)
header_row_offset	Rows down from thickness label to find column headers	Headers are the column dimension values	1
data_start_offset	Rows down from thickness label to start reading data	Data rows start here	2
price_start_column	Column where prices start (after row headers)	Row headers are in column 0, prices start in column 1	1 (Column B)

Visual Example

Excel Sheet: "US-Logos"

Column A	Column B	Column C	Column D	Column E
----------	----------	----------	----------	----------

Row 0:	[1/8 Inch,	SQ. IN.,	0.77,	...,	...]
	↑				
	Thickness label found here				
Row 1:	[-,	10,	11,	12,	13, ...]
	↑	↑			
	Skip	Column headers (square inch values) (price_start_column = 1)			
Row 2:	[10,	77.00,	84.70,	92.40,	100.10, ...]
	↑	↑			
	Row header	Price at intersection of row=10, col=10 (square inch) (price_start_column = 1)			
Row 3:	[11,	84.70,	92.40,	100.10,	107.80, ...]
	↑				
	Row header = 11				
Row 4:	[12,	92.40,	100.10,	107.80,	115.50, ...]
	...				
Row 13:	[20,	...,	...,	...,	292.60, ...]
	...				
Row 14:	[1/4 Inch,	...,	...,	...,	...]
	↑				
	Next thickness section (stop reading)				

Reading Process:

1. Find Thickness Section:

- Request: `label = "1/8 Inch"`
- Search Column A (thickness_column = 0) for "1/8 Inch"
- Finds at Row 0
- `start_row = 0`

2. Read Column Headers:

- Headers at: `start_row + header_row_offset = 0 + 1 = Row 1`
- Reads from Column B onwards: `[10, 11, 12, 13, ...]`
- These are the column dimension values

3. Find Row Value:

- Request: `row = 12` (square inch row value)
- Start reading from: `start_row + data_start_offset = 0 + 2 = Row 2`
- Check Column A of each row for the row header value
- Finds `12` at Row 4

4. Find Column Value:

- Request: `col = 15` (square inch column value)
- Check if 15 exists in headers: `[10, 11, 12, 13, ...]`
- Find index: `headers[5] = 15`
- Column index = `price_start_column + index = 1 + 5 = Column F` (index 5)

5. Get Price:

- Returns value at: Row 4, Column F
- Returns: `115.50` (example value)

6. Stop When:

- Reaches next thickness label (like "1/4 Inch")
- Or reaches end of file

For Upcharge Files

Field	Description	Example
<code>upchargeFile</code>	Filename of the upcharge Excel file	<code>"tq_upcharges_flat_cut_metal.xlsx"</code>
<code>blocks</code>	Material blocks with column mappings	See below

Block Structure:

```
"Material Name": {  
  "label": 0,    // Column index for category labels  
  "type": 1,    // Column index for finish types  
  "us": 2,      // Column index for US prices  
  "canada": 3   // Column index for Canada prices  
}
```

Example Excel Structure:

```
Row 0: [Label, Type, US, Canada, Label, Type, US, Canada, ...]  
Row 1: [Color/Finish:, Brushed - Vertical Grain, BASE PRICE, BASE PRICE, ...]  
Row 2: [-, Brushed - Horizontal Grain, 10.3, 13.39, ...]
```

Understanding FlexibleExcelParser.py

What Does This File Do?

This is the **main engine** of the system. It takes your request (like "get me the price for...") and:

1. Looks up the instructions in `config.json`
2. Opens the correct Excel file
3. Finds the exact data you need
4. Returns the price

The Main Function: `get_price()`

This is the function you'll call most often. Think of it as asking: "**Hey system, what's the price for X?**"

Function Signature

```
def get_price(category, sheet_name, params, product=None):
```

Parameters Explained (Like Talking to a Friend)

category - "Which product line are we looking at?"

- This is the top-level category in `config.json`
- Examples: `"FLAT_CUT_PVC"`, `"FLAT_CUT_METAL"`, `"LAMINATE_ON_FOAM"`
- **Think of it as:** "I want prices from the Flat Cut PVC product line"

sheet_name - "Which sheet in the Excel file?"

- This is the exact name of the sheet tab in Excel
- Examples: `"Letters PVC"`, `"US-Logos"`, `"Bars-Aluminum"`
- **Think of it as:** "Look in the 'Letters PVC' sheet"

params - "What specific price do you want?"

- A dictionary with the details needed to find the price
- Different sheet types need different parameters (see below)
- **Think of it as:** "I want US prices, thickness 1/8, size 3"

product (optional) - "Which specific product variant?"

- Only needed if the category has multiple products
- Examples: "flat_cut_metal_aluminum", "flat_cut_metal_steel"
- **Think of it as:** "Specifically the aluminum version"

What Happens Inside (Step-by-Step)

When you call `get_price()`, here's exactly what happens:

```
# Step 1: Load the configuration map
CONFIG = json.load("config.json")
# Now we know where everything is

# Step 2: Navigate to the right section
cat_cfg = CONFIG["file_structures"][category]
# Example: CONFIG["file_structures"]["FLAT_CUT_PVC"]

# Step 3: Find the Excel file
if product:
    # Category has multiple products (like FLAT_CUT_METAL has aluminum, steel,
    # etc.)
    product_cfg = cat_cfg["productlineFile"][product]
    file_name = product_cfg["file_pattern"]
    # Example: "flat_cut_metal_aluminum .xlsx"
else:
    # Category has one file directly
    file_name = cat_cfg["file_pattern"]
    # Example: "flat_cut_pvc .xlsx"

# Step 4: Get the sheet configuration
sheet_cfg = sheets[sheet_name]
sheet_type = sheet_cfg["type"]
# Example: sheet_type = "letters_price"

# Step 5: Open the Excel file
df = pd.read_excel(file_name, sheet_name=sheet_name, header=None)
# header=None means: "Don't use first row as headers, read everything as data"

# Step 6: Route to the right parser
if sheet_type == "letters_price":
    return parse_letters(df, sheet_cfg, params)
elif sheet_type == "bars":
    return parse_bars(df, sheet_cfg, params)
elif sheet_type == "logos":
    return parse_logos(df, sheet_cfg, params)
```

```
# Step 7: Return the price!
```

Return Value

The function returns the price as:

- A **number** (float or int) - the actual price
- A **string** - for special values (rare)
- Raises **ValueError** - if something can't be found

The Parser Functions (The Actual Workers)

These are the functions that do the actual work of finding prices. Each one knows how to read a specific type of sheet structure.

Parser 1: `parse_letters()` - Finding Letter Prices

What it does: Finds the price for a letter based on country, thickness, and size (column).

When to use: When you want prices for individual letters.

Parameters needed:

```
params = {  
    "country": "US",           # Which country's prices? ("US" or "Canada")  
    "thickness": "1-1/2",     # What thickness? (must match Excel exactly)  
    "column": 2               # Which size? (0 = size 1, 1 = size 2, 2 = size 3,  
    etc.)  
}
```

Detailed Step-by-Step Process:

Let's trace through an example request:

```
get_price(  
    "FLAT_CUT_PVC",  
    "Letters PVC",
```

```
{ "country": "US", "thickness": "1-1/2", "column": 2 }  
)
```

Step 1: Find the Country Section

```
# Config says: country_column = 0 (Column A)  
# Search Column A for "US"  
for i in range(len(df)):  
    cell = str(df.iloc[i, 0]).strip() # Get value from Column A, Row i  
    if cell.lower() == "us".lower(): # Case-insensitive match  
        start_row = i # Found "US" at row 1 (0-indexed)  
        break  
# Result: start_row = 1
```

Step 2: Start Reading Data Rows

```
# Config says: data_start_offset = 2  
# Start reading from: start_row + 2 = 1 + 2 = Row 3  
row = start_row + cfg["data_start_offset"] # row = 3  
  
# Config says: stop_conditions = ["US", "Canada", "nan", ""]  
stop_vals = ["us", "canada", "nan", ""] # Lowercase for comparison
```

Step 3: Search for Matching Thickness

```
while row < len(df):  
    # Check if we hit a stop condition  
    cell = str(df.iloc[row, 0]).strip() # Column A value  
    if cell.lower() in stop_vals:  
        break # Stop! We hit "Canada" or end of section  
  
    # Check if this row has our thickness  
    if cell == "1-1/2": # Exact match (case-sensitive for thickness)  
        # Found it! Get the price  
        price = df.iloc[row, cfg["price_start_column"] + column]  
        # price_start_column = 1, column = 2  
        # So: Column 1 + 2 = Column 3 (which is size 3)  
        return price # Returns: 22.00 (example)  
  
    row += 1 # Move to next row
```

Visual Representation:

Excel File: flat_cut_pvc.xlsx, Sheet: "Letters PVC"

Row	Column A	Column B	Column C	Column D	Column E
1	US ↑ Found here (start_row = 1)	-	-	-	...
2	- ↑ Skip this (header row)	1	2	3	4
3	1/8 ↑ Check: "1/8" != "1-1/2", continue	20.00	21.00	22.00	23.00
4	1/4 ↑ Check: "1/4" != "1-1/2", continue	20.00	21.00	22.00	23.00
5	1-1/2 ↑ Found! column=2 means Column C (index 2) Return: 27.00	25.00	26.00	27.00	28.00
6	3/8	30.00	31.00	32.00	33.00
...					
9	Canada ↑ Stop condition - don't read further	-	-	-	...

Parser 2: `parse_bars()` - Finding Bar Prices

What it does: Finds the price for a bar based on label, depth, and height.

When to use: When you want prices for rectangular bars.

Parameters needed:

```
params = {
    "label": 'US 1/8"',    # Bar section label (must start with "US" or "Canada")
    "depth": 36,           # Depth dimension (must match Excel value)
    "height": 5            # Height dimension (must match header value)
}
```

Detailed Step-by-Step Process:

Example request:

```
get_price(
    "FLAT_CUT_METAL",
    "Bars-Aluminum",
    {"label": 'US 1/8"', "depth": 36, "height": 5},
    "flat_cut_metal_aluminum"
)
```

Step 1: Determine Which Block (US or Canada)

```
label = 'US 1/8"'
if label.startswith("US"):
    start_col, end_col = cfg["us_columns"] # [0, 7]
    # Extract columns 0-6 (US block)
else:
    start_col, end_col = cfg["canada_columns"] # [7, 13]
    # Extract columns 7-12 (Canada block)

block = df.iloc[:, start_col:end_col] # Get only the US block columns
```

Step 2: Find the Label Row

```
# Search first column of the block for "US 1/8\"
for i in range(len(block)):
    cell = str(block.iloc[i, 0]).strip() # First column of block
    if cell == 'US 1/8':
        start_row = i # Found at row 0
        break
```

Step 3: Read Height Headers

```
# Config says: header_row_offset = 1
header_row = start_row + 1 # Row 1
heights = block.iloc[header_row, 1:7].tolist() # Columns 1-6
# Result: heights = [1, 2, 3, 4, 5]
```

Step 4: Find Matching Depth

```
# Config says: data_start_offset = 2
row = start_row + 2 # Row 2

while row < len(block):
    cell = block.iloc[row, 0] # Depth value in first column

    # Stop if we hit a text label (next section)
    if isinstance(cell, str) and cell.strip() != "":
        break

    # Check if this is our depth
    if cell == 36: # Found depth 36!
        # Find which column has our height
        height_index = heights.index(5) + 1 # height=5 is at index 4, +1 = column
```

```
price = block.iloc[row, height_index]
return price # Returns: 35.00 (example)

row += 1
```

Visual Representation:

Excel: Bars-Aluminum sheet, US Block (Columns A-G)

Row	Column A	Column B	Column C	Column D	Column E	Column F
0	US 1/8" ↑ Found label here	-	-	-	-	-
1	- ↑ Skip	1 ↑ Height headers	2	3	4	5
2	12 ↑ Check: 12 != 36, continue	10.00	11.00	12.00	13.00	14.00
3	24 ↑ Check: 24 != 36, continue	20.00	21.00	22.00	23.00	24.00
4	36 ↑ Found! height=5 is in Column F (index 5) Return: 34.00	30.00	31.00	32.00	33.00	34.00

Parser 3: `parse_logos()` - Finding Logo Prices

What it does: Finds the price for a logo from a 2D square inch table.

When to use: When you want prices for logos based on square inch dimensions.

Parameters needed:

```
params = {
    "label": "1/8 Inch", # Thickness label (must match config exactly)
    "row": 10,           # Square inch value for row lookup
    "col": 24            # Square inch value for column lookup
}
```

Detailed Step-by-Step Process:

Example request:

```
get_price(  
    "FLAT_CUT_METAL",  
    "US-Logos",  
    {"label": "1/8 Inch", "row": 10, "col": 24},  
    "flat_cut_metal_aluminum"  
)
```

Step 1: Find Thickness Section

```
# Config says: thickness_column = 0 (Column A)  
for i in range(len(df)):  
    cell = str(df.iloc[i, 0]).strip()  
    if cell == "1/8 Inch": # Exact match  
        start_row = i # Found at row 0  
        break
```

Step 2: Read Column Headers

```
# Config says: header_row_offset = 1, price_start_column = 1  
header_row = start_row + 1 # Row 1  
headers = df.iloc[header_row, 1:].tolist() # From Column B onwards  
# Result: headers = [10, 11, 12, 13, 14, ..., 24, ...]
```

Step 3: Validate Column Exists

```
col_value = 24  
if col_value not in headers:  
    raise ValueError("Column not found")  
  
# Find which column index this is  
col_index = headers.index(24) + cfg["price_start_column"]  
# headers.index(24) = 14, price_start_column = 1  
# col_index = 15 (Column P, 0-indexed)
```

Step 4: Find Matching Row

```
# Config says: data_start_offset = 2  
row = start_row + 2 # Row 2  
  
while row < len(df):  
    cell = df.iloc[row, 0] # Row header value in Column A  
  
    # Stop if we hit text (next thickness) or NaN
```



```

if isinstance(cell, str) or pd.isna(cell):
    break

# Check if this is our row value
if cell == 10: # Found row 10!
    price = df.iloc[row, col_index] # Row 2, Column 15
    return price # Returns: 84.70 (example)

row += 1

```

Visual Representation:

Excel: US-Logos sheet

Row	Column A	Column B	Column C	...	Column P (15)
0	1/8 Inch ↑ Found thickness label	SQ. IN.	0.77	...	
1	- ↑ Skip	10 ↑ Column headers	11	...	24
2	10 ↑ Found! row=10	77.00 ↑ col=10	84.70 ↑ col=11	...	292.60 ↑ col=24
		Return value at row=2, col=15: 292.60			

2. `parse_bars(df, cfg, params)`

Extracts bar prices based on label, depth, and height.

Parameters:

- `df`: pandas DataFrame of the Excel sheet
- `cfg`: Sheet configuration from config.json
- `params`: Dictionary with:
 - `label` (str): Bar section label (e.g., 'US 1/8"', 'Canada 1/4"')
 - `depth` (int/float): Depth value
 - `height` (int/float): Height value

Process:

1. Determines US or Canada block based on label prefix
2. Extracts the column range for that country
3. Searches for the label in the first column

4. Reads height headers from `header_row_offset` row
5. Searches for matching depth value
6. Returns price at intersection of depth and height

Example:

```
get_price(  
    "FLAT_CUT_METAL",  
    "Bars-Aluminum",  
    {"label": 'US 1/8"', "depth": 36, "height": 5},  
    "flat_cut_metal_aluminum"  
)
```

3. `parse_logos(df, cfg, params)`

Extracts logo prices from a 2D square inch table.

Parameters:

- `df`: pandas DataFrame of the Excel sheet
- `cfg`: Sheet configuration from config.json
- `params`: Dictionary with:
 - `label` (str): Thickness label (e.g., "1/8 Inch", "1/4 Inch")
 - `row` (int/float): Square inch value for row lookup
 - `col` (int/float): Column header value for column lookup

Process:

1. Searches for thickness label in `thickness_column`
2. Reads column headers from `header_row_offset` row
3. Validates that requested column exists
4. Searches for matching row value (square inches)
5. Returns price at intersection of row and column

Example:

```
get_price(  
    "FLAT_CUT_METAL",  
    "US-Logos",  
    {"label": "1/8 Inch", "row": 10, "col": 24},  
    "flat_cut_metal_aluminum"  
)
```

Understanding upcharges.py

What Does This File Do?

This file handles **additional charges** (upcharges) that apply on top of base prices. For example:

- A base letter might cost \$20
- But if you want a special finish like "Brushed - Vertical Grain", there might be an upcharge of \$5
- Total price = $20 + 5 = \$25$

However, some finishes might say "BASE PRICE" which means no extra charge - just use the base price.

Why Separate from Main Parser?

Upcharges are stored in a **different Excel file** with a **different structure**:

- Multiple materials side-by-side (Aluminum, Stainless Steel, etc.)
- Categories and finish types in rows
- Special handling for "BASE PRICE" values
- Different lookup logic (category + finish type matching)

The Main Function: `get_upcharge()`

This function finds additional charges for special finishes, mounting options, etc.

Function Signature

```
def get_upcharge(product_line, material, category, finish_type, country):
```

Parameters Explained

product_line - "Which product line's upcharges?"

- Same as category in `get_price()`
- Examples: "FLAT_CUT_METAL", "GEMLEAF_LAMINATE"
- **Think of it as:** "I want upcharges for Flat Cut Metal products"

material - "Which material block?"

- The material name from the config blocks
- Examples: "Aluminum", "Stainless Steel", "Brass/Bronze"
- **Think of it as:** "Specifically for aluminum material"

category - "What type of upcharge?"

- The category label in Excel
- Examples: "Color/Finish:", "Mounting Option"
- **Important:** May need trailing colon (:) - check Excel file
- **Think of it as:** "I want a finish upcharge" or "I want a mounting option upcharge"

finish_type - "Which specific option?"

- The exact finish or option name
- Examples: "Brushed - Vertical Grain", "Double Faced Tape"
- **Think of it as:** "Specifically the brushed vertical grain finish"

country - "US or Canada price?"

- "US" or "Canada"
- **Think of it as:** "Get the US upcharge price"

Return Values

The function can return three different things:

1. **A number** (float) - The upcharge amount

```
return 10.50 # Add $10.50 to base price
```

2. **The string "BASE PRICE"** - No upcharge, use base price

```
return "BASE PRICE" # No extra charge
```

3. **None** - No upcharge found or invalid

```
return None # Treat as $0 upcharge
```

Detailed Step-by-Step Process

Let's trace through an example:

```
get_upcharge(  
    product_line="FLAT_CUT_METAL",  
    material="Aluminum",  
    category="Color/Finish:",  
    finish_type="Brushed - Vertical Grain",  
    country="US"  
)
```

Step 1: Load Config and Find File

```
# Load config  
CONFIG = json.load("config.json")  
  
# Navigate to product line  
product_cfg = CONFIG["file_structures"]["FLAT_CUT_METAL"]  
  
# Get upcharge file name  
file_name = product_cfg["upchargeFile"]  
# Result: "tq_upcharges_flat_cut_metal.xlsx"  
  
# Get material block structure  
blocks = product_cfg["upchargeFileStructure"]["blocks"]  
cols = blocks["Aluminum"]  
# Result: {"label": 0, "type": 1, "us": 2, "canada": 3}
```

Step 2: Read Excel File

```
df = pd.read_excel(file_name, header=None)  
# header=None means: "Read everything as data, no header row"
```

Step 3: Search for Matching Row

```

for row in range(len(df)):
    # Get values from the material's columns
    label = str(df.iloc[row, cols["label"]]).strip()      # Column 0
    option = str(df.iloc[row, cols["type"]]).strip()      # Column 1

    # Normalize for comparison (case-insensitive)
    label_lower = label.lower()
    option_lower = option.lower()
    category_lower = category.lower()

    # Check if label is valid (category header or sub-row)
    valid_label = (
        label == "-" or                # Sub-row under category
        label_lower == category_lower or # Category header row
        label_lower == "nan" or        # Empty/invalid
        label == ""                    # Empty
    )

    # Check if this row matches
    if option_lower == finish_type.lower() and valid_label:
        # Found matching row!
        # Get price from US or Canada column
        if country.upper() == "US":
            price = df.iloc[row, cols["us"]]      # Column 2
        else:
            price = df.iloc[row, cols["canada"]]  # Column 3

        # Handle special values
        price_str = str(price).strip().upper()

        if price_str == "BASE PRICE":
            return "BASE PRICE"

        if price_str in ["", ".", "NAN", "-"]:
            return None

        return float(price) # Return the upcharge amount

# If we get here, no match was found
return None

```

Visual Example:

Excel File: tq_upcharges_flat_cut_metal.xlsx

Aluminum Block (Columns A-D)

A	B
Label	Type
Color/	Brushed - Vertical
Finish:	Grain
	BASE PRICE

Stainless Steel Block (Columns E-H)

E	F
Label	Type
Color/	Brushed - Vertical
Finish:	Grain
	12.50

(US col)	(US col)
- Brushed - Horizontal Grain 10.30 (US col)	- Brushed - Horizontal Grain 15.20 (US col)
Mounting Double Faced Tape Option 5.00 (US col)	Mounting Double Faced Tape Option 6.00 (US col)

Matching Logic Explained:

The function matches rows using **two conditions**:

1. Finish Type Must Match:

```
option_lower == finish_type.lower()
# "brushed - vertical grain" == "brushed - vertical grain" ✓
```

2. Label Must Be Valid:

- **Option A:** Label is "-" (sub-row under a category)

```
Row: [-, Brushed - Horizontal Grain, 10.30, ...]
```

- **Option B:** Label matches category (category header row)

```
Row: [Color/Finish:, Brushed - Vertical Grain, BASE PRICE, ...]
```

- **Option C:** Label is empty/invalid (rare edge case)

Why This Logic?

Excel structure has two row types:

- **Header rows:** [Color/Finish:, Brushed - Vertical Grain, ...] - category in label column
- **Sub-rows:** [-, Brushed - Horizontal Grain, ...] - "-" in label column, finish type in type column

Both can have valid upcharges, so we check for either pattern.

Example:

```
get_upcharge(  
    product_line="FLAT_CUT_METAL",  
    material="Stainless Steel",  
    category="Mounting Option",  
    finish_type="Double Faced Tape",  
    country="US"  
)
```

Excel Structure

The upcharge Excel file has multiple material blocks side by side:

Columns:	A	B	C	D	E	F
G	H					
Row 0:	Label	Type	US	Canada	Label	Type
US	Canada					
Row 1:	Color/	Brushed - Vertical Grain	BASE	BASE	Color/	Brushed -
Vertical Grain	BASE	BASE				
	Finish:	PRICE	PRICE	Finish:	PRICE	PRICE
Row 2:	-	Brushed - Horizontal	10.3	13.39	-	Brushed -
Horizontal	12.5	15.2				
	Grain			Grain		
Row 3:	Mounting Double Faced Tape		5.0	6.5	Mounting Double	
Faced Tape	6.0	7.8				
	Option			Option		

Block Mapping:

- **Aluminum:** Columns 0-3 (A-D)
- **Stainless Steel:** Columns 4-7 (E-H)
- **Brass/Bronze:** Columns 8-11 (I-L)
- **Corten:** Columns 12-15 (M-P)

Matching Logic

The function matches rows based on:

1. **Finish Type Match:** `option.lower() == finish_type.lower()`

2. Label Match: One of:

- `label == "-"` (sub-row under a category)
- `label.lower() == category.lower()` (category header row)
- `label == ""` or `label.lower() == "nan"` (empty/invalid)

Special Cases:

- Category labels may have trailing colons (e.g., "Color/Finish:")
- Sub-rows use "-" as the label
- Some values may be "BASE PRICE" instead of a number

Complete Usage Examples

This section provides real-world examples with detailed explanations of what each line does and why.

Example 1: Get a Simple Letter Price

Scenario: You want to know the price for a PVC letter, 1-1/2 inch thick, size 3, in the US.

```
from FlexibleExcelParser import get_price

# Call the function with your requirements
price = get_price(
    category="FLAT_CUT_PVC",           # Product line: Flat Cut PVC
    sheet_name="Letters PVC",         # Sheet name in Excel
    params={
        "country": "US",              # US prices (not Canada)
        "thickness": "1-1/2",         # Thickness value (must match Excel exactly)
        "column": 2                   # Column 2 = size 3 (0=size1, 1=size2,
2=size3)
    }
)

print(f"Price: ${price}") # Output: Price: $27.00
```

What happens behind the scenes:

1. System looks up "FLAT_CUT_PVC" in config.json

2. Finds file: "flat_cut_pvc .xlsx"
 3. Opens sheet: "Letters PVC"
 4. Finds "US" section in Column A
 5. Searches for thickness "1-1/2" in Column A
 6. Gets value from Column C (column index 2)
 7. Returns: 27.00
-

Example 2: Get a Bar Price

Scenario: You want the price for an aluminum bar, US market, 1/8 inch, depth 36, height 5.

```
from FlexibleExcelParser import get_price

price = get_price(
    category="FLAT_CUT_METAL",           # Product line: Flat Cut Metal
    sheet_name="Bars-Aluminum",         # Sheet name
    params={
        "label": 'US 1/8"',             # Bar section label (must start with "US" or
"Canada")
        "depth": 36,                    # Depth dimension (must match Excel value)
        "height": 5                     # Height dimension (must match header
value)
    },
    product="flat_cut_metal_aluminum"   # Product variant (required for
FLAT_CUT_METAL)
)

print(f"Bar Price: ${price}") # Output: Bar Price: $34.00
```

What happens behind the scenes:

1. System looks up "FLAT_CUT_METAL" → "flat_cut_metal_aluminum" product
 2. Finds file: "flat_cut_metal_aluminum .xlsx"
 3. Opens sheet: "Bars-Aluminum"
 4. Label starts with "US" → uses US block (columns 0-6)
 5. Finds "US 1/8" label in Column A
 6. Reads height headers: [1, 2, 3, 4, 5]
 7. Finds depth 36 in Column A
 8. Gets price at intersection of depth=36 and height=5
 9. Returns: 34.00
-

Example 3: Get a Logo Price

Scenario: You want the price for a logo, 1/8 inch thick, with dimensions 10×24 square inches.

```
from FlexibleExcelParser import get_price

price = get_price(
    category="FLAT_CUT_METAL",
    sheet_name="US-Logos",           # US logos sheet
    params={
        "label": "1/8 Inch",        # Thickness label (must match config
    exactly)
        "row": 10,                  # Square inch value for row (first
    dimension)
        "col": 24                   # Square inch value for column (second
    dimension)
    },
    product="flat_cut_metal_aluminum"
)

print(f"Logo Price: ${price}") # Output: Logo Price: $292.60
```

What happens behind the scenes:

1. System finds "US-Logos" sheet in config
2. Opens Excel file and sheet
3. Searches Column A for "1/8 Inch" label
4. Reads column headers: [10, 11, 12, ..., 24, ...]
5. Finds row with value 10 in Column A
6. Gets value at intersection of row=10 and col=24
7. Returns: 292.60

Note: The `row` and `col` parameters represent the two dimensions of the logo. Think of it as a grid where you look up the price at coordinates (row, col).

Example 4: Get an Upcharge

Scenario: You want to know the upcharge for "Brushed - Vertical Grain" finish on aluminum.

```

from upcharges import get_upcharge

upcharge = get_upcharge(
    product_line="FLAT_CUT_METAL",      # Product line
    material="Aluminum",                 # Material block name
    category="Color/Finish:",            # Category (note the colon!)
    finish_type="Brushed - Vertical Grain", # Specific finish
    country="US"                         # US or Canada
)

# Handle the different return types
if upcharge == "BASE PRICE":
    print("Base price applies - no upcharge")
elif upcharge is None:
    print("No upcharge found")
else:
    print(f"Upcharge: ${upcharge}") # Output: Upcharge: $BASE PRICE

```

What happens behind the scenes:

1. System loads config and finds upcharge file: "tq_upcharges_flat_cut_metal.xlsx"
2. Gets Aluminum block columns: {label: 0, type: 1, us: 2, canada: 3}
3. Searches for row where:
 - Type column = "Brushed - Vertical Grain"
 - Label column = "Color/Finish:" OR "-"
4. Gets value from US column (column 2)
5. Value is "BASE PRICE" → returns string "BASE PRICE"

Important Notes:

- Category may need a trailing colon (:) - check your Excel file
- "BASE PRICE" means no extra charge
- **None** means treat as \$0 upcharge

Example 5: Complete Price Calculation (Real-World Scenario)

Scenario: Calculate the total price for a letter with a special finish.

```

from FlexibleExcelParser import get_price
from upcharges import get_upcharge

```

```

# Step 1: Get the base price
base_price = get_price(
    "FLAT_CUT_METAL",
    "Letters - Aluminum",
    {
        "country": "US",
        "thickness": "1/8",
        "column": 3 # Size 4
    },
    "flat_cut_metal_aluminum"
)
print(f"Base Price: ${base_price}") # Output: Base Price: $22.00

# Step 2: Get the upcharge for special finish
upcharge = get_upcharge(
    "FLAT_CUT_METAL",
    "Aluminum",
    "Color/Finish:",
    "Brushed - Vertical Grain",
    "US"
)
print(f"Upcharge: {upcharge}") # Output: Upcharge: BASE PRICE

# Step 3: Calculate total price
if upcharge == "BASE PRICE":
    # "BASE PRICE" means no extra charge, use base price only
    total = base_price
elif upcharge is None:
    # None means no upcharge found, treat as $0
    total = base_price
else:
    # upcharge is a number, add it to base price
    total = base_price + upcharge

print(f"Total Price: ${total}") # Output: Total Price: $22.00

```

Complete Output:

```

Base Price: $22.00
Upcharge: BASE PRICE
Total Price: $22.00

```

What this example teaches:

- How to combine base prices and upcharges
 - How to handle the three different upcharge return types
 - Real-world pricing calculation logic
-

Example 6: Error Handling

Scenario: What to do when things go wrong.

```
from FlexibleExcelParser import get_price

try:
    price = get_price(
        "FLAT_CUT_PVC",
        "Letters PVC",
        {"country": "US", "thickness": "1-1/2", "column": 2}
    )
    print(f"Price: ${price}")
except ValueError as e:
    print(f"Error finding price: {e}")
    # Possible errors:
    # - "Country not found"
    # - "Thickness not found"
    # - "Column index out of range"
```

Common Errors and What They Mean:

Error Message	What It Means	How to Fix
"Country not found"	Couldn't find "US" or "Canada" in the sheet	Check country_labels in config, verify Excel has country labels
"Thickness not found"	Couldn't find the thickness value you specified	Check exact format in Excel (e.g., "1-1/2" vs "1 1/2")
"Bar label not found"	Couldn't find the bar section label	Verify label exactly matches Excel (including quotes: 'US 1/8')
"Logo thickness label not found"	Couldn't find the thickness section	Check label matches config thickness_labels exactly
"Square inch value not found"	Couldn't find the row or column value	Verify the square inch values exist in the table
"Column index out of range"	Column number is too high	Check Excel has enough columns, verify column parameter

Example 7: Working with Multiple Products

Scenario: Get prices for different materials in the same product line.

```
from FlexibleExcelParser import get_price

# Aluminum letters
aluminum_price = get_price(
    "FLAT_CUT_METAL",
    "Letters - Aluminum",
    {"country": "US", "thickness": "1/8", "column": 2},
    "flat_cut_metal_aluminum" # Specify product
)

# Stainless Steel letters (same product line, different product)
steel_price = get_price(
    "FLAT_CUT_METAL",
    "Letters - Stainless Steel",
    {"country": "US", "thickness": "1/8", "column": 2},
    "flat_cut_metal_steel" # Different product
)

print(f"Aluminum: ${aluminum_price}, Steel: ${steel_price}")
```

Key Point: When a category has multiple products (like FLAT_CUT_METAL has aluminum, steel, corten, etc.), you **must** provide the **product** parameter.

Example 8: Batch Processing

Scenario: Get prices for multiple items at once.

```
from FlexibleExcelParser import get_price

# List of items to price
items = [
    {"thickness": "1/8", "column": 2},
    {"thickness": "1/4", "column": 2},
    {"thickness": "3/8", "column": 2},
]

prices = []
for item in items:
    price = get_price(
        "FLAT_CUT_PVC",
        "Letters PVC",
        {"country": "US", **item} # Spread item params
    )
    prices.append(price)
```

```
)
prices.append(price)
print(f"Thickness {item['thickness']}: ${price}")

# Calculate total
total = sum(prices)
print(f"Total for all items: ${total}")
```

Output:

```
Thickness 1/8: $21.00
Thickness 1/4: $21.00
Thickness 3/8: $26.00
Total for all items: $68.00
```

Common Issues and Solutions

Issue 1: "Country not found"

Cause: Country label in Excel doesn't match the search term (case-sensitive after lowercasing) **Solution:** Check `country_labels` in config and ensure Excel has exact match

Issue 2: "Thickness not found"

Cause: Thickness value in params doesn't exactly match Excel value **Solution:** Verify exact format in Excel (e.g., "`1-1/2`" vs "`1 1/2`")

Issue 3: "Logo thickness label not found"

Cause: Thickness label doesn't match config `thickness_labels` **Solution:** Ensure label exactly matches one in `thickness_labels` array

Issue 4: Upcharge returns None

Cause:

- Category label mismatch (may have trailing colon)
- Finish type doesn't match exactly
- Row structure doesn't match expected pattern **Solution:**
- Check if category needs colon: "Color/Finish:" vs "Color/Finish"
- Verify finish type matches Excel exactly
- Check if row uses "-" for sub-rows

Issue 5: Column index out of range

Cause: `price_start_column` or column number exceeds Excel columns **Solution:**
Verify Excel file has enough columns and config is correct

Configuration Best Practices

1. **Column Indices:** Always use 0-based indexing (0 = Column A, 1 = Column B, etc.)
 2. **Row Offsets:** Count from the reference row (country label or thickness label)
 3. **Stop Conditions:** Include all possible values that indicate end of data
 4. **Case Sensitivity:** Use lowercase comparisons for text matching
 5. **Special Values:** Document special values like "BASE PRICE" in config comments
-

File Dependencies

```
config.json
├── FlexibleExcelParser.py (reads file_structures)
└── upcharges.py (reads file_structures → upchargeFileStructure)

Excel Files:
├── flat_cut_metal_aluminum.xlsx (used by FlexibleExcelParser)
├── flat_cut_pvc.xlsx (used by FlexibleExcelParser)
├── tq_upcharges_flat_cut_metal.xlsx (used by upcharges.py)
└── ... (other product files)
```

Notes

- All Excel files are read with `header=None` to preserve exact row/column positions
 - Column and row indices are 0-based
 - String comparisons are case-insensitive where appropriate
 - Empty cells, NaN values, and special strings are handled gracefully
 - The system is designed to be flexible - new product lines can be added by updating `config.json`
-

Quick Reference Guide

For Beginners: What Do I Need to Know?

1. The Three Main Concepts

`config.json` = The Map

- Tells the system where to find data
- You edit this when Excel files change structure
- No Python knowledge needed - just JSON

`FlexibleExcelParser.py` = The Price Finder

- Gets base prices for letters, bars, and logos
- You call `get_price()` with parameters
- Returns a number (the price)

`upcharges.py` = The Extra Charges Finder

- Gets additional charges for finishes/options
- You call `get_upcharge()` with parameters
- Returns a number, "BASE PRICE", or None

2. Quick Decision Tree

Need a price?

|

- └ Is it a letter? → Use `get_price()` with type "letters_price"
 - └ Need: country, thickness, column
- └ Is it a bar? → Use `get_price()` with type "bars"
 - └ Need: label, depth, height
- └ Is it a logo? → Use `get_price()` with type "logos"
 - └ Need: label (thickness), row, col
- └ Need an upcharge? → Use `get_upcharge()`
 - └ Need: product_line, material, category, finish_type, country

3. Common Parameter Patterns

Letters:

```
params = {  
    "country": "US",           # Always "US" or "Canada"  
    "thickness": "1/8",       # Must match Excel exactly  
    "column": 2               # 0=size1, 1=size2, 2=size3, etc.  
}
```

Bars:

```
params = {  
    "label": 'US 1/8"',       # Must start with "US" or "Canada"  
    "depth": 36,              # Must match Excel value  
    "height": 5               # Must match header value  
}
```

Logos:

```
params = {  
    "label": "1/8 Inch",      # Must match config thickness_labels  
    "row": 10,                # Square inch row value  
    "col": 24                 # Square inch column value  
}
```

Upcharges:

```
get_upcharge(  
    product_line="FLAT_CUT_METAL",  
    material="Aluminum",      # Material block name  
    category="Color/Finish:", # May need colon!
```

```
finish_type="Brushed - Vertical Grain", # Exact match
country="US"
)
```

4. Key Things to Remember

✓ DO:

- Check Excel file for exact format (thickness values, labels, etc.)
- Use exact sheet names from config.json
- Handle "BASE PRICE" return value from upcharges
- Use try/except for error handling
- Verify column indices (0-based: 0=A, 1=B, 2=C, etc.)

✗ DON'T:

- Guess parameter values - check Excel first
- Forget the **product** parameter for multi-product categories
- Ignore case sensitivity for thickness values
- Assume upcharge is always a number (could be "BASE PRICE" or None)

5. When Things Go Wrong

"Country not found" → Check if Excel has "US" or "Canada" in the country column → Verify country_labels in config

"Thickness not found" → Check exact format: "1-1/2" vs "1 1/2" vs "1.5" → Open Excel and verify the exact value

"Column index out of range" → Excel doesn't have enough columns → Check column parameter isn't too high

Upcharge returns None → Category might need colon: "Color/Finish:" vs "Color/Finish" → Finish type might not match exactly → Check if row uses "-" for sub-rows

6. Understanding Return Values

get_price() always returns:

- A number (float/int) - the price

- Raises ValueError - if not found

get_upcharge() can return:

- A number - add this to base price
- "BASE PRICE" - no upcharge, use base price only
- None - no upcharge found, treat as \$0

7. Complete Example Template

```
from FlexibleExcelParser import get_price
from upcharges import get_upcharge

# 1. Get base price
try:
    base = get_price(
        "CATEGORY_NAME",
        "Sheet Name",
        {"country": "US", "thickness": "1/8", "column": 2},
        "product_name" # If needed
    )
except ValueError as e:
    print(f"Error: {e}")
    base = 0

# 2. Get upcharge
upcharge = get_upcharge(
    "CATEGORY_NAME",
    "Material",
    "Category:",
    "Finish Type",
    "US"
)

# 3. Calculate total
if upcharge == "BASE PRICE":
    total = base
elif upcharge is None:
    total = base
else:
    total = base + upcharge

print(f"Total: ${total}")
```

Summary

This system provides a **configuration-driven approach** to reading Excel pricing data. Instead of hardcoding file paths and cell positions in your code, you:

1. **Define the structure** in `config.json` (the map)
2. **Call simple functions** in Python (the interface)
3. **Get prices** automatically (the result)

The system handles:

- ☒ Different Excel file structures
- ☒ Multiple product lines
- ☒ US and Canada pricing
- ☒ Base prices and upcharges
- ☒ Special values like "BASE PRICE"

Key Takeaway: The config file is your friend. When Excel files change, update the config - your Python code stays the same!

Next Steps

1. **Try the examples** - Run them with your actual Excel files
2. **Check your config** - Verify file names and sheet names match
3. **Test edge cases** - Try different thicknesses, sizes, etc.
4. **Add error handling** - Use try/except blocks in production code
5. **Extend as needed** - Add new product lines by updating config.json

Happy coding! 🚀